HIGH PERFORMANCE COMPUTING FOR SCIENCE AND ENGINEERING

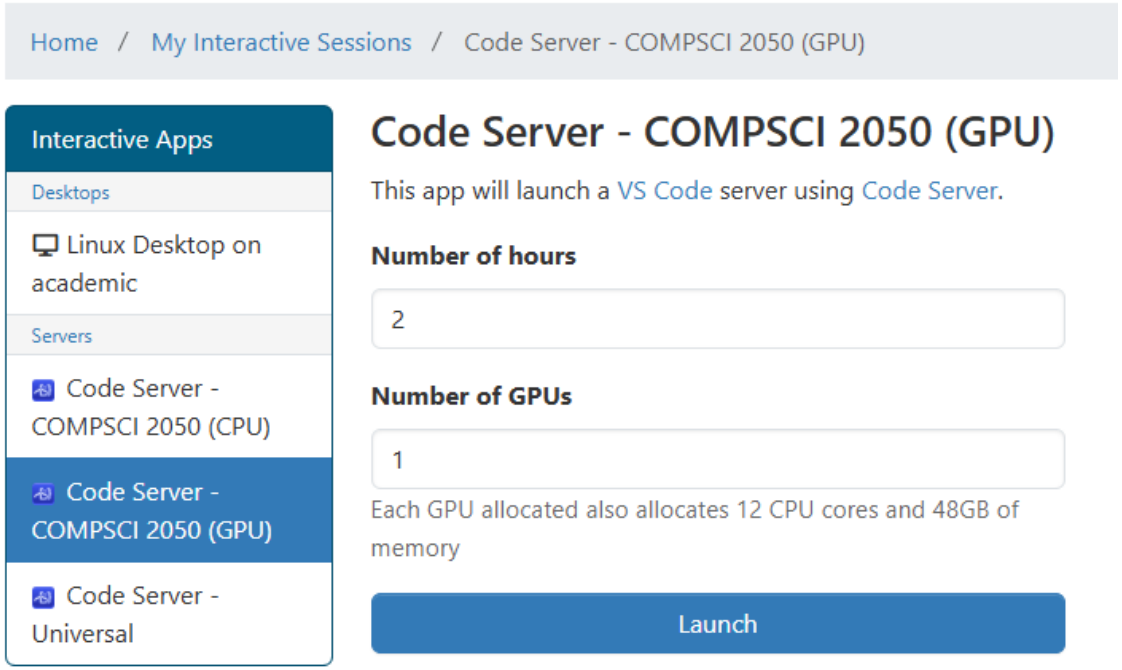
Harvard University
CS 2050

Lecture 12

March 5, 2026

SETUP

- Request a GPU



CS-2050 / homework-3

Code Issues Pull requests Actions Projects Wiki Security Insights Settings

homework-3

Internal template

Edit Pins | Watch 0 | Fork 0 | Star 0 | Use this template

main | 1 Branch | 0 Tags | Go to file t | Add file | Code

laz380 and Chuck Witt public release. 35621a7 · 4 days ago 1 Commits

question-1	public release.	4 days ago
question-2	public release.	4 days ago
question-3	public release.	4 days ago
question-4	public release.	4 days ago
question-5	public release.	4 days ago
question-6	public release.	4 days ago
question-7	public release.	4 days ago
README.md	public release.	4 days ago

README

Homework 3

Welcome to Homework 3. This assignment is due by 11:59pm on **Thursday March 12.** Begin with the setup instructions in the next section, and don't forget the submission instructions at the bottom.

About

No description, website, or topics provided.

Readme
Activity
Custom properties
0 stars
0 watching
0 forks

Releases

No releases published
Create a new release

Contributors 2

wcw398 Chuck Witt
laz380 Laura Zichi

Languages

C++ 78.9% Cuda 14.5% Shell 6.0% CMake 0.6%

CS-2050 / questions-and-answers

Type / to search

Code Issues Pull requests Actions Projects Wiki Security Insights Settings

questions-and-answers

Public

Edit Pins | Watch 0 | Fork 0 | Star 0

main | Go to file t | Code

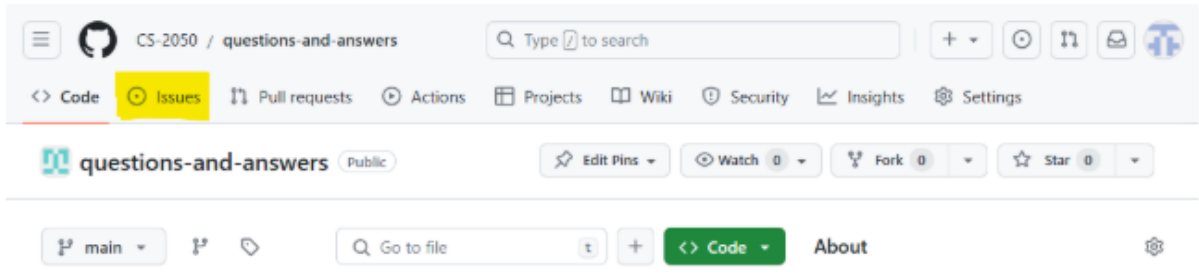
wcw398 update readme 4be70a2 · 17 hours ago 4 Commits

README.md update readme 17 hours ago

README

Questions and Answers

Please create issues in this repository to ask questions about the course.



About

Ask your CS 2050 questions here.

sites.google.com/g.harvard.edu/cs-2050

Readme

Activity

Custom properties

0 stars

0 watching

0 forks

Releases

No releases published

Create a new release

project Internal

Edit Pins Watch 0

main 1 Branch 0 Tags Go to file t Add file Code

laz380 and Chuck Witt draft release. 31d14b3 · 2 minutes ago 1 Commits

README.md draft release. 2 minutes ago

README

CS 2050 Final Project

> ⚠ **Warning**
>
> This is a DRAFT of the final project specification, as of *March 5, 2026*. Please review and ask any clarification questions over the next two weeks. Based on feedback, we will refine and release the final specification at the end of the Spring Recess.

Overview

The final project requires you to apply the high-performance computing techniques learned in class to a computational task of your choice. The project will be completed **individually**. You will begin with a serial version of your algorithm, and then develop parallel versions using OpenMP, MPI, and CUDA. You will analyze its performance in scaling as well as with profiling tools. Finally, you will explore an additional topic beyond the core course material.

BITS AND BYTES

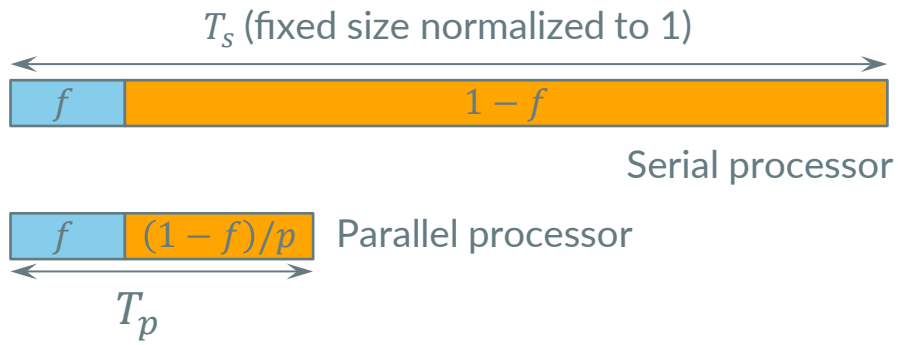
How many bits (and bytes) are used to store a single precision floating point number?

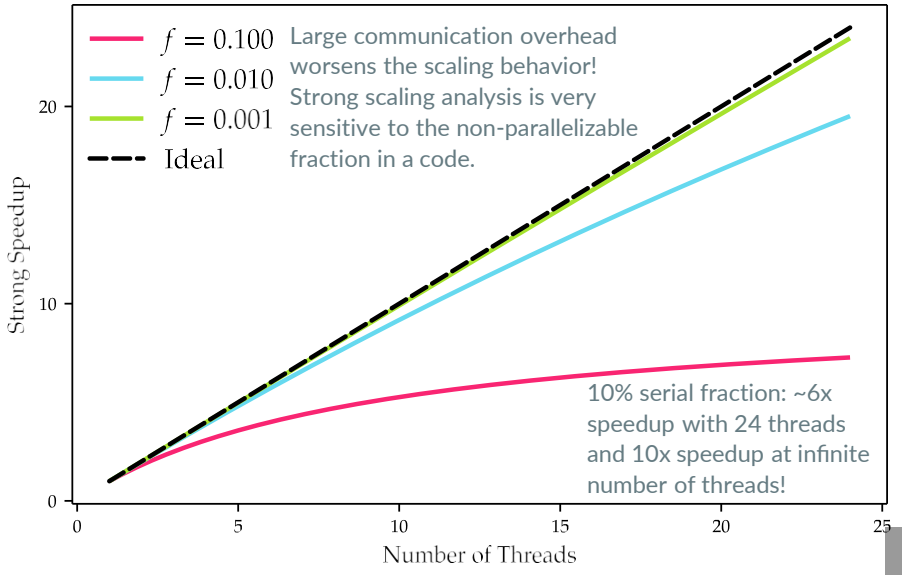
- 16 bits (2 bytes)
- 32 bits (4 bytes)
- 64 bits (8 bytes)
- 128 bits (16 bytes)

STRONG SCALING

- A speedup analysis formulated using Amdahl's Law is called ***strong scaling***.
- It assumes the problem size is fixed (defined by the sequential problem) and you add more parallel processors to *arrive at the solution faster*.
- The serial fraction in a code becomes the dominant factor in this model. *Serial fraction is hard to determine in general.*
- Achieving good strong scaling speedup for a large scale system with thousands of processors is ***very hard***.
- Strong scaling analysis makes sense for systems with few processors and negligible communication overhead →shared memory.

$$S_p = \frac{T_s}{T_p} = \frac{1}{f + \frac{1-f}{p}}$$



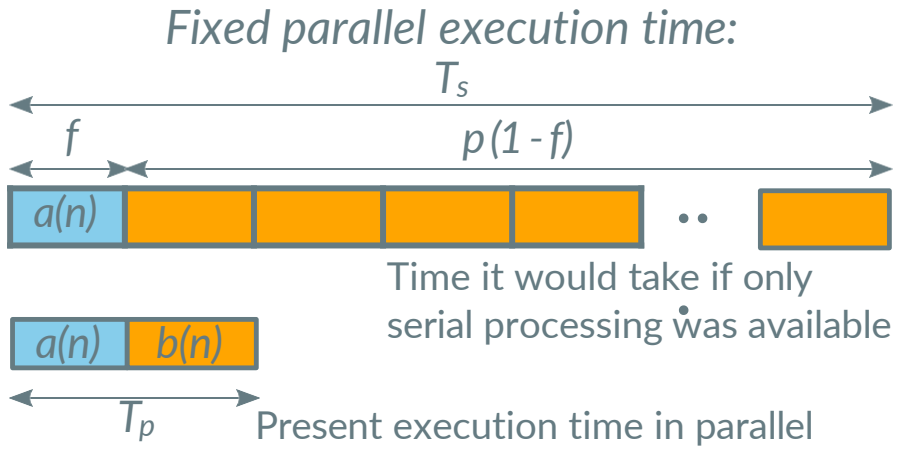


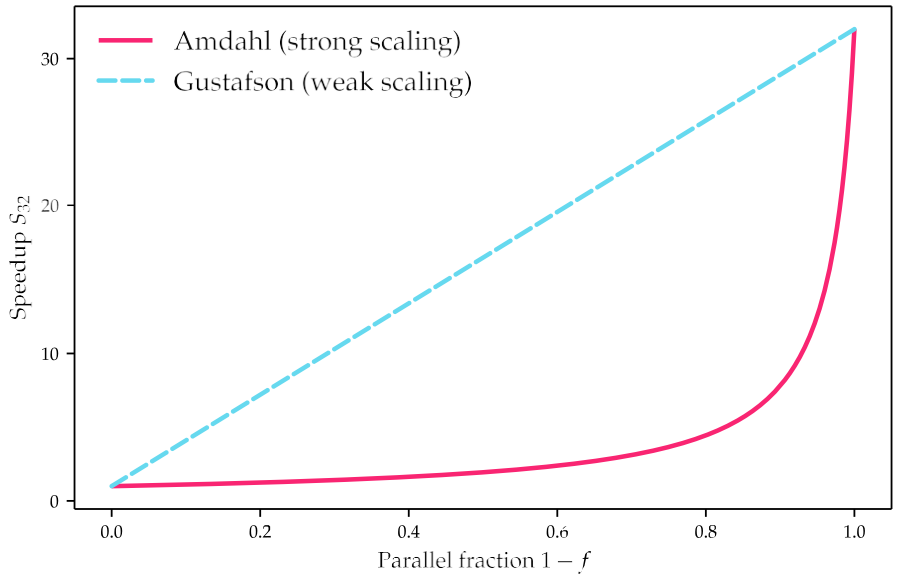
WEAK SCALING

- A speedup analysis formulated with this model is called ***weak scaling***.
- It assumes the execution time per processor (the work per processor) is fixed and you add more parallel processors to increase the problem size to arrive at a larger solution *in the same time* as it would take for one processor working on a p-times smaller problem.
- The argument is equivalent to a work size of the parallel problem that is p times the serial problem, i.e., $W_p = pW_s$. The speedup is

$$S_p = \frac{W_p/T_p}{W_s/T_s} = \frac{pT_s}{T_p} = \frac{pT_s}{T_s + T_o}$$

where this assumes a diminishing serial fraction for $n \longrightarrow \infty$ but non-zero parallel overhead as $T_o = T_o(n; p)$.





Observable speedup for 32 processors as a function of the *parallel fraction* in a code, expressed in terms of Amdahl's Law and Gustafson's Law.

EXAMPLE 1

NSIGHT SYSTEMS

NVIDIA Nsight Systems

NVIDIA Nsight™ Systems is a system-wide performance analysis tool designed to visualize an application's algorithms, identify the largest opportunities to optimize, and tune to scale efficiently across any quantity or size of CPUs and GPUs, from large servers to our smallest systems-on-a-chip (SoCs).

EXAMPLE 2
NSIGHT COMPUTE

NVIDIA Nsight Compute

NVIDIA Nsight™ Compute is an interactive profiler for CUDA® and NVIDIA OptiX™ that provides detailed performance metrics and API debugging via a user interface and command-line tool. Users can run guided analysis and compare results with a customizable and data-driven user interface, as well as post-process and analyze results in their own workflows.

Get started

NVIDIA Nsight Compute is also available as part of the CUDA Toolkit.

CS 2050 Challenges

Reminder: Your pseudonym can be found with `challenge_pseudonym` on the course cluster.

Last Update: Thursday, Mar 05, 1:51:27 PM

Leaderboards

challenge_0

0 ranked submissions

No results yet.